Formattazione di date e orari

Corso di Laurea in Ingegneria Informatica

Anno accademico 2025/2026

Prof. ENRICO DENTI
Dipartimento di Informatica – Scienza e Ingegneria (DISI)

FORMATTATORI PER DATE E ORARI

Java

- Tempo fa abbiamo visto i formattatori numerici
  - classe **`java.text.NumberFormat`**
  - costruzione tramite metodi factory della forma **`get**(…)`** distinti per *numeri*, *percentuali* e *valute*
  - schema d'uso: ***`formatter`*`.format(valore)`**
- I formattatori per date e orari
  - sono definiti in **`java.time.format.DateTimeFormatter`**
  - costruzione tramite metodi factory della forma **`of**(…)`** distinti per *orari*, *date, date con orario,* etc.
  - duplice schema d'uso:

    `formatter.format(`*`valore`*`)` come sopra

    `valore.format(`*`formatter`*`)` novità, schema invertito

FORMATTATORI PER DATE E ORARI

Java

- La costruzione avviene tramite tre metodi factory fondamentali:
  - per DATE: **`ofLocalizedDate`**
  - per ORARI: **`ofLocalizedTime`**
  - per DATE CON ORARIO: **`ofLocalizedDateTime`**
- Se i formati prestabiliti non soddisfano, se ne possono definire altri personalizzati, tramite il metodo **`ofPattern`**
- Per specificare una cultura locale, anziché includerla come argomento, occorre *specializzare il formattatore dopo averlo prodotto*, invocando su esso il metodo **`withLocale(`*`locale`*`)`**
  - solo **`ofPattern`** offre anche una versione a due argomenti che accetta direttamente un ***`locale`***

STILI DI FORMATTAZIONE

- I formattatori per date e orari consentono di formattare i valori con quattro diversi *stili,* espressi dall'enumerativo **`FormatStyle`**
  - stile SHORT (breve)
  - stile MEDIUM (medio)
  - stile LONG (lungo)
  - stile FULL (completo)
- Più precisamente:
  - le date ammettono tutti i 4 formati *breve*, *medio, lungo, completo*
  - gli orari ammettono solo i 2 formati *breve, medio*
  - le date con orario ammettono quindi 4x2 formati (4 per la parte data, 2 per la parte orario)

STILI DI FORMATTAZIONE: ESEMPI

- Formattazione di date (cultura italiana)
  - **SHORT** *gg/mm/aa* 18/03/15
  - **MEDIUM** *gg siglamese aaaa* 18 mar 2015
  - **LONG** *gg mese aaaa* 18 marzo 2015
  - **FULL** *giorno gg mese aaaa* mercoledì 18 marzo 2015
- Formattazione di orari (cultura italiana)
  - **SHORT** *hh:mm* 18:37
  - **MEDIUM** *hh:mm:ss* 18:37:24
  - **LONG** e **FULL** *non* sono supportati: se usati, causano errore
- Naturalmente, tutti i formati dipendono dalla *cultura locale* per quanto attiene all'ordine, al nome, al formato e ai separatori delle singole voci.

ESEMPIO: FORMATTAZIONE DI `LocalDate`

Sperimentiamo i formattatori nei quattro diversi stili:

Java

```
DateTimeFormatter formatterShort =
    DateTimeFormatter.ofLocalizedDate(FormatStyle.SHORT);
DateTimeFormatter formatterMedium =
    DateTimeFormatter.ofLocalizedDate(FormatStyle.MEDIUM);
DateTimeFormatter formatterLong =
    DateTimeFormatter.ofLocalizedDate(FormatStyle.LONG);
DateTimeFormatter formatterFull =
    DateTimeFormatter.ofLocalizedDate(FormatStyle.FULL);

LocalDate d = LocalDate.now();

System.out.println(d.format(formatterShort));
System.out.println(d.format(formatterMedium));
System.out.println(d.format(formatterLong));
System.out.println(d.format(formatterFull));
```

ESEMPIO: FORMATTAZIONE DI `LocalDate`

Output con cultura locale italiana (default):

```
04/03/26
4 mar 2026
4 marzo 2026
mercoledì 4 marzo 2026
```

Short

Medium

Long

Full

Java

Osserva le convenzioni italiane (non è così in altre culture):

- nello <u>short</u>, formattano l'anno su due cifre
- nel <u>full</u>, non inseriscono virgole o altri separatori

E se volessimo la short, ma con l'anno su quattro cifre..?

```
04/03/2026
```

Non è previsto! E se servisse..?

- dovremmo definire un *pattern personalizzato!*

FORMATTAZIONE DI `LocalDate` CON PATTERN PERSONALIZZATO

Se i formati prestabiliti non soddisfano, se ne possono definire altri, tramite il metodo factory **`ofPattern`**

Java

```
DateTimeFormatter myFormatter =
    DateTimeFormatter.ofPattern("dd/MM/yyyy");

LocalDate d = LocalDate.now();

System.out.println(d.format(myFormatter));
```

04/03/2026

Pattern Letters and Symbols

Symbol	Meaning	Presentation	Examples
G	era	text	AD; Anno Domini; A
u	year	year	2004; 04
y	year-of-era	year	2004; 04
D	day-of-year	number	189
M/L	month-of-year	number/text	7; 07; Jul; July; J

The count of pattern letters determines the format.

Text: The text style is determined based on the number of pattern letters used. Less than 4 pattern letters will use the `short form`. Exactly 4 pattern letters will use the `full form`. Exactly 5 pattern letters will use the `narrow form`. Pattern letters 'L', 'c', and 'q' specify the stand-alone form of the text styles.

Number: If the count of letters is one, then the value is output using the minimum number of digits and without padding. Otherwise, the count of digits is used as the width of the output field, with the value zero-padded as necessary. The following pattern letters have constraints on the count of letters. Only one letter of 'c' and 'F' can be specified. Up to two letters of 'd', 'H', 'h', 'K', 'k', 'm', and 's' can be specified. Up to three letters of 'D' can be specified.

Number/Text: If the count of pattern letters is 3 or greater, use the Text rules above. Otherwise use the Number rules above.

FORMATTAZIONE DI `LocalDate` CON CULTURE LOCALI DIVERSE

Java

- Per specificare una cultura locale si invoca sul formattatore già creato il metodo **`withLocale(`*`locale`*`)`**

```
DateTimeFormatter.ofLocalizedDate(…)
                 .withLocale(Locale.CANADA)
```

OSSERVA: è un (altro) esempio di *fluent interface*

ESEMPIO: FORMATTAZIONE DI `LocalDate`

Output con cultura locale italiana (default):

```
04/03/26
4 mar 2026
4 marzo 2026
mercoledì 4 marzo 2026
```

Short
Medium
Long
Full
Java

Output con cultura locale inglese UK e USA:

```
04/03/2026
4 Mar 2026
4 March 2026
Wednesday 4 March 2026
```

Short
Medium
Long
Full
Java

```
3/4/26
Mar 4, 2026
March 4, 2026
Wednesday, March 4, 2026
```

Short
Medium
Long
Full
Java

ESEMPIO: FORMATTAZIONE DI `LocalDate`

Output con cultura locale italiana (default):

```
04/03/26
4 mar 2026
4 marzo 2026
mercoledì 4 marzo 2026
```

Java

Output con cultura locale francese e franco-canadese

```
04/03/2026
4 mars 2026
4 mars 2026
mercredi 4 mars 2026
```

Java

In febbraio e altri mesi dal nome più lungo:

Medium → **`févr.`**

Long → **`février`**

```
2026-03-04
4 mars 2026
4 mars 2026
mercredi 4 mars 2026
```

ESEMPIO COMPLESSIVO: FORMATTAZIONE DI `LocalDate`

```
import java.time.*;
import java.time.format.*;
import java.util.Locale;

public class Main {
   public static void main(String[] args) {

       Locale[] locales = { Locale.US, Locale.ITALY, Locale.UK,
                            Locale.FRANCE, Locale.CANADA_FRENCH };

       var formatterShort = DateTimeFormatter.ofLocalizedDate(FormatStyle.SHORT);
       var formatterMedium = DateTimeFormatter.ofLocalizedDate(FormatStyle.MEDIUM);
       var formatterLong = DateTimeFormatter.ofLocalizedDate(FormatStyle.LONG);
       var formatterFull = DateTimeFormatter.ofLocalizedDate(FormatStyle.FULL);

       var d = LocalDate.of(2026,3,4);

       for (Locale loc: locales){
            System.out.println(loc);
            System.out.println(d.format(formatterShort.withLocale(loc)));
            System.out.println(d.format(formatterMedium.withLocale(loc)));
            System.out.println(d.format(formatterLong.withLocale(loc)));
            System.out.println(d.format(formatterFull.withLocale(loc)));
            System.out.println();
       }
   }
}
```

ESEMPIO COMPLESSIVO: FORMATTAZIONE DI `LocalDate`

```
import java.time.*;
import java.time.format.*;
import java.util.Locale;

public class Main {
   public static void main(String[] args) {

       Locale[] locales = { Locale.US, Locale.ITALY, Locale.UK,
                            Locale.FRANCE, Locale.CANADA_FRENCH };

       var formatterShort = DateTimeFormatter.ofLocalizedDate(Forma
       var formatterMedium = DateTimeFormatter.ofLocalizedDate(For
       var formatterLong = DateTimeFormatter.ofLocalizedDate(Forma
       var formatterFull = DateTimeFormatter.ofLocalizedDate(Forma

       var d = LocalDate.of(2026,3,4);

       for (Locale loc: locales){
            System.out.println(loc);
            System.out.println(d.format(formatterShort.withLocale(
            System.out.println(d.format(formatterMedium.withLocale
            System.out.println(d.format(formatterLong.withLocale(l
            System.out.println(d.format(formatterFull.withLocale(l
            System.out.println();
       }
   }
}
```

```
en_US
3/4/26
Mar 4, 2026
March 4, 2026
Wednesday, March 4, 2026

it_IT
04/03/26
4 mar 2026
4 marzo 2026
mercoledì 4 marzo 2026

en_GB
04/03/2026
4 Mar 2026
4 March 2026
Wednesday, 4 March 2026

fr_FR
04/03/2026
4 mars 2026
4 mars 2026
mercredi 4 mars 2026

fr_CA
2026-03-04
4 mars 2026
4 mars 2026
mercredi 4 mars 2026
```

FORMATTATORI PER ORARI LOCALI

- I formattatori per orari locali sono analoghi a quelli per date
  - metodi **`ofLocalizedTime`**
- ma accettano *solo due* formati di stile:
  - **`SHORT`** *hh:mm* 18:37
  - **`MEDIUM`** *hh:mm:ss* 18:37:24
  - **`LONG`** e **`FULL`** NON sono supportati: se usati, causano errore

```
DateTimeFormatter formatterShort =
    DateTimeFormatter.ofLocalizedTime(FormatStyle.SHORT);
DateTimeFormatter formatterMedium =
    DateTimeFormatter.ofLocalizedTime(FormatStyle.MEDIUM);

LocalTime t = LocalTime.now();

System.out.println(t.format(formatterShort));
System.out.println(t.format(formatterMedium));
```

Java

FORMATTATORI PER ORARI LOCALI

Esempi: cultura locale italiana, UK, USA

```
18:37
18:37:41
```

Short

Medium

Java

Italia

UK

Francia

```
6:37 PM
6:37:41 PM
```

Short

Medium

Java

USA

Esempi: il caso multiculturale del Canada

```
6:37 p.m.
6:37:41 p.m.
```

Canada

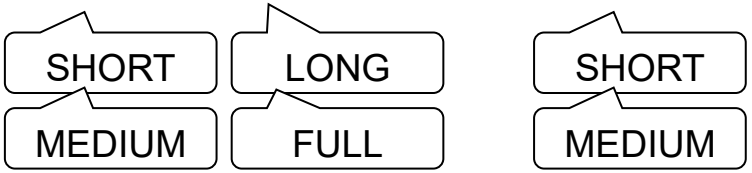
```
18 h 37
18 h 37 min 41 s
```

Canada francese

FORMATTATORI PER DATE/ORARI LOCALI

- I formattatori omnicomprensivi per date/orari sono analoghi
  - Java: metodi `ofLocalizedDateTime`
  - doppia specifica di stili: una per le data, una per l'orario
  - data: ammessi tutti i quattro formati di stile
  - orario: possibili solo i due formati SHORT e MEDIUM
- Totale: 8 possibili combinazioni

```
DateTimeFormatter.ofLocalizedDateTime(
                    dateStyle, timeStyle);
```



FORMATTATORI PER DATE/ORARI LOCALI

Output con cultura locale italiana (default):

altri esempi con altri mesi & anni

```
04/03/26, 18:37
04/03/26, 18:37:41
4 mar 2026, 18:37
4 mar 2026, 18:37:41
4 marzo 2026, 18:37
4 marzo 2026, 18:37:41
mercoledì 4 marzo 2026, 18:37
mercoledì 4 marzo 2026, 18:37:41
```

Tutte le 8 combinazioni

Java

Come sarà negli altri paesi...? ☺

FORMATTATORI PER DATE/ORARI ASSOLUTI

- I formattatori gestiscono anche *date/orari assoluti*
  - RICORDA: una data/ora assoluto è tale *perché contiene una specifica* che lo àncora a un *ben preciso punto* sul globo terrestre
- In Java, una data/orario assoluta può essere espresso:
  - specificando un fuso orario: istanze di `ZonedDateTime`
  - specificando un offset da GMT: istanze di `OffsetDateTime`
  - NB: non esistono classi assolute di «sola data» o «solo orario»: proprio in quanto assolute, sono sempre composte di date + orario
- IMPORTANTE: quelle due classi rappresentano la stessa cosa in modo diverso, non due concetti diversi!
  - conseguenza: *i formattatori agiscono indifferentemente su entrambe,* con output molto simile (cambia solo l'indicazione della timezone)

DATE/ORARI ASSOLUTI: RECAP

- Esempi di date/orari assoluti

```
var odt = OffsetDateTime.of(2025,3,4,18,37,41,0,
                                    ZoneOffset.ofHours(1));
var zdt = ZonedDateTime.of(2025,3,4,18,37,41,0,
                                    ZoneId.of("CET"));
```

Altre possibilità: **`GMT+1`**, **`Europe/Paris`**, etc.

  - la prima specifica *un offset da GMT*
  - la seconda specifica *un fuso orario* (per nome o offset)

- If the zone ID equals 'Z', the result is `ZoneOffset.UTC`.
- If the zone ID consists of a single letter, the zone ID is invalid and `DateTimeException` is thrown.
- If the zone ID starts with '+' or '-', the ID is parsed as a `ZoneOffset` using `ZoneOffset.of(String)`.
- If the zone ID equals 'GMT', 'UTC' or 'UT' then the result is a `ZoneId` with the same ID and rules equivalent to `ZoneOffset.UTC`.
- If the zone ID starts with 'UTC+', 'UTC-', 'GMT+', 'GMT-', 'UT+' or 'UT-' then the ID is a prefixed offset-based ID. The ID is split in two, with a two or three letter prefix and a suffix starting with the sign. The suffix is parsed as a `ZoneOffset`. The result will be a `ZoneId` with the specified UTC/GMT/UT prefix and the normalized offset ID as per `ZoneOffset.getId()`. The rules of the returned `ZoneId` will be equivalent to the parsed `ZoneOffset`.
- All other IDs are parsed as region-based zone IDs. Region IDs must match the regular expression `[A-Za-z][A-Za-z0-9~/._+-]+` otherwise a `DateTimeException` is thrown. If the zone ID is not in the configured set of IDs, `ZoneRulesException` is thrown. The detailed format of the region ID depends on the group supplying the data. The default set of data is supplied by the IANA Time Zone Database (TZDB). This has region IDs of the form '{area}/{city}', such as 'Europe/Paris' or 'America/New_York'. This is compatible with most IDs from `TimeZone`.

- Z - for UTC
- `+h`
- `+hh`
- `+hh:mm`
- `-hh:mm`
- `+hhmm`
- `-hhmm`
- `+hh:mm:ss`
- `-hh:mm:ss`
- `+hhmmss`
- `-hhmmss`

FORMATTATORI PER DATE/ORARI ASSOLUTI

- Lo schema generale resta identico: si possono richiedere formattatori analoghi ai precedenti, per:
  - sola data `ofLocalizedDate`
  - solo orario `ofLocalizedTime`
  - entrambi `ofLocalizedDateTime`
- L'unica cosa che cambia è che `ZonedDateTime` ammette 4 varianti di stile anche per l'orario, perché in stile LONG e FULL vengono mostrati il fuso orario e l'ora solare/legale, che dipendono dalla zona
  - tutto il resto (incluso `OffsetDateTime`) funziona coma nei casi precedenti, quindi con 4 varianti di stile per la data e soltanto 2 per l'orario → 8 combinazioni possibili

FORMATTATORI PER DATE/ORARI ASSOLUTI

```
ZonedDateTime zdt = ZonedDateTime.now();
```

```
19:27
19:27:13
19:27:13 CET
19:27:13 Ora standard dell'Europa centrale
```

con **ofLocalizedTime** (solo orario)

Italia

Java

```
19:27
19:27:13
19:27:13 CET
19:27:13 Central European Standard Time
```

UK

Java

```
19 h 27
19 h 27 min 13 s
19 h 27 min 13 s CET
19 h 27 min 13 s heure normale d'Europe centrale
```

Canada francese

Java

```
19:27
19:27:13
19:27:13 CET
19:27:13 heure normale d'Europe centrale
```

Francia

Java

FORMATTATORI PER DATE/ORARI ASSOLUTI

```
OffsetDateTime odt = OffsetDateTime.now();
```

```
19:27
19:27:13
```

con **ofLocalizedTime** (solo orario)

Italia

Java

```
19:27
19:27:13
```

OffsetDateTime supporta solo i formati SHORT e MEDIUM per l'orario

UK

Java

```
19 h 27
19 h 27 min 13 s
```

Canada francese

Java

```
19:27
19:27:13
```

Francia

Java

ORA LEGALE E FUSI ORARI IN `ZonedDateTime`

- Tenere presente che:
  - la specifica del fuso orario può essere data in vari modi, alcuni più precisi di altri (es. «Europe/Rome», «CET», «GMT+1», etc.): in particolare, la forma GMT±n *non* specifica il nome della zona
  - è il formattatore a emettere la sigla (formato **`LONG`**) o la descrizione (formato **`FULL`**) adattata nel caso sia in vigore l'*ora legale*
  - ESEMPIO: col fuso orario CET, il 4 marzo 2026 il formato LONG emette la sigla «CET», ma il 4 aprile 2026 la sigla «CEST»; analoga modifica avviene con le descrizioni nel formato FULL

```
19:27
19:27:13
19:27:13 CET
19:27:13 Ora standard dell'Europa centrale
```

4/3/2026

```
19:27
19:27:13
19:27:13 CEST
19:27:13 Ora legale dell'Europa centrale
```

4/4/2026

Java

ORA LEGALE E FUSI ORARI IN `ZonedDateTime`

```
var zdt = ZonedDateTime.of(2026,4,4,18,37,41,0,  ZoneId.of("Europe/Rome") );
var zdt = ZonedDateTime.of(2026,4,4,18,37,41,0,  ZoneId.of("GMT+1") );
var zdt = ZonedDateTime.of(2026,4,4,18,37,41,0,  ZoneId.of("CET") );
```

- la forma «Europe/Rome» è la più precisa
- sigle come «CET», «GMT», etc. sono accettate per retro-compatibilità, ma non tutte (es. mancano le americane), perché spesso ambigue
- il formato GMT±n è meno ricco: si sa la zona, ma non il suo nome → conseguente output meno dettagliato
- il formattatore fa del suo meglio per emettere la sigla (formato **`LONG`**) o la descrizione (formato **`FULL`**) più precisa in base alle info che ha

```
18:37:41 GMT+01:00                          // formato LONG con ZoneId.of("GMT+1")
18:37:41 GMT+01:00                          // formato FULL con ZoneId.of("GMT+1")
19:37:41 CEST                               // formato LONG con ZoneId.of("CET")
19:37:41 Ora legale dell'Europa centrale    // formato FULL con ZoneId.of("CET")
```

ORA LEGALE E FUSI ORARI IN ZonedDateTime

```
var zdt = ZonedDateTime.of(..., ZoneId.of("America/Los_Angeles"));
System.out.println(zdt.format(DateTimeFormatter.ofLocalizedTime(FormatStyle.LONG)));
System.out.println(zdt.format(DateTimeFormatter.ofLocalizedTime(FormatStyle.FULL)));
// la stessa ora riferita a un diverso fuso orario
System.out.println(zdt.format(DateTimeFormatter.ofLocalizedTime(
                FormatStyle.LONG).withZone(ZoneId.of("CET"))));
System.out.println(zdt.format(DateTimeFormatter.ofLocalizedTime(
                FormatStyle.FULL).withZone(ZoneId.of("Europe/Paris"))));
```

IT

```
18:37:41 PDT
18:37:41 Ora legale del Pacifico USA
03:37:41 CEST
03:37:41 Ora legale dell'Europa centrale
```

UK

```
18:37:41 GMT-07:00
18:37:41 Pacific Daylight Time
03:37:41 CEST
03:37:41 Central European Summer Time
```

US

```
6:37:41 PM PDT
6:37:41 PM Pacific Daylight Time
3:37:41 AM CEST
3:37:41 AM Central European Summer Time
```

CAN

```
6:37:41 p.m. PDT
6:37:41 p.m. Pacific Daylight Saving Time
3:37:41 a.m. CEST
3:37:41 a.m. Central European Summer Time
```

FR

```
18:37:41 PDT
18:37:41 heure d'été du Pacifique nord-américain
03:37:41 CEST
03:37:41 heure d'été d'Europe centrale
```

CAN

```
18 h 37 min 41 s HAP
18 h 37 min 41 s heure avancée du Pacifique
03 h 37 min 41 s CEST
03 h 37 min 41 s heure avancée de l'Europe
centrale
```

FORMATTATORI PER DATE/ORARI ASSOLUTI

- Sono inoltre disponibili *formatter standard (ISO)* per leggere/scrivere date e orari in *formato interoperabile*
  - adatti a interazioni machine-to-machine, *non verso umani*
  - ce ne sono più di una dozzina!

Predefined Formatters

Predefined Formatters

Formatter	Description	Example
ofLocalizedDate(dateStyle)	Formatter with date style from the locale	'2011-12-03'
ofLocalizedTime(timeStyle)	Formatter with time style from the locale	'10:15:30'
ofLocalizedDateTime(dateTimeStyle)	Formatter with a style for date and time from the locale	'3 Jun 2008 11:05:30'
ofLocalizedDateTime(dateStyle,timeStyle)	Formatter with date and time styles from the locale	'3 Jun 2008 11:05'
BASIC_ISO_DATE	Basic ISO date	'20111203'
ISO_LOCAL_DATE	ISO Local Date	'2011-12-03'
ISO_OFFSET_DATE	ISO Date with offset	'2011-12-03+01:00'
ISO_DATE	ISO Date with or without offset	'2011-12-03+01:00'; '2011-12-03'
ISO_LOCAL_TIME	Time without offset	'10:15:30'
ISO_OFFSET_TIME	Time with offset	'10:15:30+01:00'
ISO_TIME	Time with or without offset	'10:15:30+01:00'; '10:15:30'
ISO_LOCAL_DATE_TIME	ISO Local Date and Time	'2011-12-03T10:15:30'
ISO_OFFSET_DATE_TIME	Date Time with Offset	'2011-12-03T10:15:30+01:00'
ISO_ZONED_DATE_TIME	Zoned Date Time	'2011-12-03T10:15:30+01:00[Europe/Paris]'
ISO_DATE_TIME	Date and time with ZoneId	'2011-12-03T10:15:30+01:00[Europe/Paris]'
ISO_ORDINAL_DATE	Year and day of year	'2012-337'
ISO_WEEK_DATE	Year and Week	'2012-W48-6'
ISO_INSTANT	Date and Time of an Instant	'2011-12-03T10:15:30Z'
RFC_1123_DATE_TIME	RFC 1123 / RFC 822	'Tue, 3 Jun 2008 11:05:30 GMT'

FORMATTATORI ISO

```
OffsetDateTime odt = OffsetDateTime.now();
```

```
ZonedDateTime zdt = ZonedDateTime.now();
```

```
17:04:44.9189596+01:00
```

con **ISO OFFSET TIME**

Java

```
2026-03-15T17:10:34.0637975
```

con **ISO LOCAL DATE TIME**

Java

```
LocalDate d = LocalDate.now();
```

```
20250215
```

con **BASIC_ISO_DATE**

Java

FORMATTATORI PERSONALIZZATI

- Non vi piacciono/bastano i formattatori standard?
  - vorreste *il simbolo di € davanti* invece che in fondo nelle valute?
  - vorreste magari poter fare il *parsing del solo nome del mese* (es. "Maggio") *nella lingua della cultura locale* prescelta, ottenendone il corrispondente indice (es. 5)?
- In Java, potete farvi un *formattatore personalizzato* tramite il metodo `DateTimeFormatter.ofPattern(pattern)`
  - ovviamente, a patto di studiarsi i (molti) simboli del `pattern`... ☺
  - vedremo un esempio tra poco (→ slide 33)

PARSING DI DATE E ORARI

- I formattatori Java supportano anche la conversione da stringa a data/ora, tramite una serie di metodi **`parse`**
  - OCCHIO: come nel caso dei valori numerici, tentando di usare **`parse`** in un vostro programma *otterrete errore di compilazione* perché la gestione degli errori è obbligatoria e voi ancora non sapete farla
  - possiamo però fare alcuni esperimenti usando **`jshell`** in cui la gestione degli errori è gestita già dall'ambiente interattivo
- Sono possibili due approcci alternativi:
  - [scomodo] chiedere al formattatore di fare lui il parsing della stringa
    il risultato è un «oggetto temporale», da convertire però nel tipo «giusto»
    → richiede un'operazione accessoria
  - [comodo] chiedere a LocalDate/Time di fare lei il parsing della stringa tramite un metodo factory, passandole il formattatore da usare

PARSING DI DATE E ORARI
Primo approccio (scomodo)

- I formattatori creati da **`DateTimeFormatter.ofXXX`** espongono vari metodi **`parse`**
  - MA il metodo **`parse`** basico restituisce un **`TemporalAccessor`**, che è una sorta di «contenitore generico» per oggetti «temporali»
  - il risultato va quindi convertito esplicitamente nel *tipo di oggetto atteso* tramite i metodi-factory **`from`** presenti in **`LocalDate`**, **`LocalTime`**, **`LocalDateTime`**

Java

```
jshell> import java.time.*
jshell> import java.time.format.*
jshell> import java.time.temporal.*

jshell> var fmt = DateTimeFormatter.ofLocalizedDate(FormatStyle.SHORT)
                  .withLocale(Locale.ITALY)

jshell> var t = fmt.parse("12/02/26")

jshell> LocalDate d = LocalDate.from(t)
d ==> 2026-02-12
```

Restituisce un TemporalAccessor…

…da usare poi per costruire la data col metodo-factory **`from`**

PARSING DI DATE E ORARI
Primo approccio

– Un altro esempio con **`LocalDateTime`**

```
jshell> var fmt = DateTimeFormatter.ofLocalizedDateTime(
                                    FormatStyle.SHORT,FormatStyle.SHORT)
                              .withLocale(Locale.ITALY)

jshell> var tp = fmt.parse("12/02/26, 12:40")

jshell> LocalDate d = LocalDate.from(tp)
d ==> 2026-02-12

jshell> LocalTime t = LocalTime.from(tp)
h ==> 12:40

jshell> LocalDateTime dt = LocalDateTime.from(tp)
dt ==> 2026-02-12T12:40
```

Restituisce il solito TemporalAccessor

Essendo una data/ora, si possono estrarre a piacimento o solo la data, o solo l'ora, o entrambe

Java

– Nei casi banali, ovviamente, si può anche fare tutto insieme:

```
jshell> var dt = LocalDateTime.from(fmt.parse("12/02/26, 12:40"))
```

PARSING DI DATE E ORARI
Secondo approccio (più comodo)

- In alternativa, si possono sfruttare i metodi **`parse`** esposti da **`LocalDate`**, **`LocalTime`**, etc.
  - in questo caso, il formattatore che governa la conversione viene passato come (secondo) argomento al metodo **`parse`** stesso

Java

```
jshell> var dt = LocalDateTime.parse("12/02/26, 12:30", fmt)
dt ==> 2026-02-12T12:30

jshell> var d = LocalDate.parse("12/02/26, 12:30", fmt)
d ==> 2026-02-12

jshell> var t = LocalTime.parse("12/02/26, 12:30", fmt)
t ==> 12:30
```

Si invoca il metodo **`parse`** della classe che interessa

Il formattatore è il secondo argomento del metodo **`parse`**

  - NB: il formattatore dev'essere coerente con la stringa passata → quindi nell'esempio sopra adatto per **`LocalDateTime`**
  - però, il metodo **`parse`** invocato può anche essere quello di **`LocalDate`** o **`LocalTime`**, se interessa solo la data o solo l'ora

FORMATTATORI PERSONALIZZATI PER DATE/ORARI

Pattern Letters and Symbols

Symbol	Meaning	Presentation	Examples
G	era	text	AD; Anno Domini; A
u	year	year	2004; 04
y	year-of-era	year	2004; 04
D	day-of-year	number	189
M/L	month-of-year	number/text	7; 07; Jul; July; J
d	day-of-month	number	10
g	modified-julian-day	number	2451334
Q/q	quarter-of-year	number/text	3; 03; Q3; 3rd quarter
Y	week-based-year	year	1996; 96
w	week-of-week-based-year	number	27
W	week-of-month	number	4
E	day-of-week	text	Tue; Tuesday; T
e/c	localized day-of-week	number/text	2; 02; Tue; Tuesday; T
F	day-of-week-in-month	number	3
a	am-pm-of-day	text	PM
h	clock-hour-of-am-pm (1-12)	number	12
K	hour-of-am-pm (0-11)	number	0
k	clock-hour-of-day (1-24)	number	24
H	hour-of-day (0-23)	number	0
m	minute-of-hour	number	30
s	second-of-minute	number	55
S	fraction-of-second	fraction	978
A	milli-of-day	number	1234
n	nano-of-second	number	987654321
N	nano-of-day	number	1234000000
V	time-zone ID	zone-id	America/Los_Angeles; Z; -08:30
v	generic time-zone name	zone-name	Pacific Time; PT
v	generic time-zone name	zone-name	Pacific Time; PT
z	time-zone name	zone-name	Pacific Standard Time; PST
O	localized zone-offset	offset-O	GMT+8; GMT+08:00; UTC-08:00
X	zone-offset 'Z' for zero	offset-X	Z; -08; -0830; -08:30; -083015; -08:30:15
x	zone-offset	offset-x	+0000; -08; -0830; -08:30; -083015; -08:30:15
Z	zone-offset	offset-Z	+0000; -0800; -08:00
p	pad next	pad modifier	1
'	escape for text	delimiter	
''	single quote	literal	'
[	optional section start		
]	optional section end		
#	reserved for future use		
{	reserved for future use		
}	reserved for future use		

Java

The count of pattern letters determines the format.

Text: The text style is determined based on the number of pattern letters used. Less than 4 pattern letters will use the `short form`. Exactly 4 pattern letters will use the `full form`. Exactly 5 pattern letters will use the `narrow form`. Pattern letters 'L', 'c', and 'q' specify the stand-alone form of the text styles.

Number: If the count of letters is one, then the value is output using the minimum number of digits and without padding. Otherwise, the count of digits is used as the width of the output field, with the value zero-padded as necessary. The following pattern letters have constraints on the count of letters. Only one letter of 'c' and 'F' can be specified. Up to two letters of 'd', 'H', 'h', 'K', 'k', 'm', and 's' can be specified. Up to three letters of 'D' can be specified.

Number/Text: If the count of pattern letters is 3 or greater, use the Text rules above. Otherwise use the Number rules above.

FORMATTATORI PERSONALIZZATI
Esempio: parsing del nome del mese

- Problema: come risalire alla *costante enumerativa mese* partendo dal *nome* del mese stesso?
  - ad esempio, come ottenere **`Month.MAY`** partendo da **`"May"`** ?
- Se il nome del mese è *in inglese*, basta **`Month.valueOf`**
  - purché la stringa sia scritta ESATTAMENTE come si aspetta **`Month`**, ossia *tutta in maiuscolo*

```
jshell> Month.valueOf("May")
|  Exception java.lang.IllegalArgumentException: No enum constant
h.May
|        at Enum.valueOf (Enum.java:240)
|        at Month.valueOf (Month.java:106)
|        at (#87:1)

jshell> Month.valueOf("MAY")
$88 ==> MAY

jshell> Month.valueOf("may")
|  Exception java.lang.IllegalArgumentException: No enum constant
h.may
|        at Enum.valueOf (Enum.java:240)
|        at Month.valueOf (Month.java:106)
|        at (#89:1)
```

Java

FORMATTATORI PERSONALIZZATI
Esempio: parsing del nome del mese

- Ma se il nome del mese è *in italiano*??
  - il metodo **`Month.valueOf`** non funziona più!
- Definire un nostro enumerativo **`Mese`** per i mesi in italiano *non avrebbe alcun senso,* sarebbe una *follia*
  - sarebbe un doppione
  - ma soprattutto *non sarebbe integrato nella libreria* ***`java.time`*** che, quindi, non riuscirebbe a usarlo
  - ergo, *farebbe impazzire* perché bisognerebbe *continuamente convertire valori dell'uno in valori dell'altro*: DELIRIO!
- Molto meglio definire un *formattatore personalizzato,* tramite il metodo **`DateTimeFormatter.ofPattern`**

FORMATTATORI PERSONALIZZATI
Esempio: parsing del nome del mese

Patterns for Formatting and Parsing

Patterns are based on a simple sequence of letters and symbols. A pattern is used to create a Formatter u
'3 Dec 2011'. A formatter created from a pattern can be used as many times as necessary, it is immutable

For example:

```
LocalDate date = LocalDate.now();
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy MM dd");
String text = date.format(formatter);
LocalDate parsedDate = LocalDate.parse(text, formatter);
```

All letters 'A' to 'Z' and 'a' to 'z' are reserved as pattern letters. The following pattern letters are

Symbol	Meaning	Presentation	Examples
G	era	text	AD; Anno Domini; A
u	year	year	2004; 04
y	year-of-era	year	2004; 04
D	day-of-year	number	189
M/L	month-of-year	number/text	7; 07; Jul; July; J
d	day-of-month	number	10
Q/q	quarter-of-year	number/text	3; 03; Q3; 3rd quarter
Y	week-based-year	year	1996; 96
w	week-of-week-based-year	number	27
W	week-of-month	number	4
E	day-of-week	text	Tue; Tuesday; T
e/c	localized day-of-week	number/text	2; 02; Tue; Tuesday; T
F	week-of-month	number	3
a	am-pm-of-day	text	PM
h	clock-hour-of-am-pm (1-12)	number	12
K	hour-of-am-pm (0-11)	number	0
k	clock-hour-of-am-pm (1-24)	number	0
H	hour-of-day (0-23)	number	0
m	minute-of-hour	number	30
s	second-of-minute	number	55
S	fraction-of-second	fraction	978
A	milli-of-day	number	1234

A	milli-of-day	number	1234
n	nano-of-second	number	987654321
N	nano-of-day	number	1234000000
V	time-zone ID	zone-id	America/Los_Angeles; Z; -08:30
z	time-zone name	zone-name	Pacific Standard Time; PST
O	localized zone-offset	offset-O	GMT+8; GMT+08:00; UTC-08:00;
X	zone-offset 'Z' for zero	offset-X	Z; -08; -0830; -08:30; -083015; -08:30:15;
x	zone-offset	offset-x	+0000; -08; -0830; -08:30; -083015; -08:30:15;
Z	zone-offset	offset-Z	+0000; -0800; -08:00;
p	pad next	pad modifier	1
'	escape for text	delimiter	
''	single quote	literal	'
[	optional section start		
]	optional section end		
#	reserved for future use		
{	reserved for future use		
}	reserved for future use		

Java

Less than 4 pattern letters will use the short form.
Exactly 4 pattern letters will use the full form.

D	day-of-year	number	189
M/L	month-of-year	number/text	7; 07; Jul; July; J
d	day-of-month	number	10

FORMATTATORI PERSONALIZZATI
Esempio: parsing del nome del mese

- Poiché vogliamo riconoscere il nome completo del mese
  - ad esempio "maggio", *non "mag" o altre forme abbreviate*
- ci serve un formattatore in formato FULL → pattern **"MMMM"**

Java

```
public class MyDateUtils {
  public static int getMonth(String s, Locale lc){
    DateTimeFormatter f = DateTimeFormatter.ofPattern("MMMM", lc);
     java.time.temporal.TemporalAccessor t =
            f.parse(s.toLowerCase());

     return t.get(
            java.time.temporal.ChronoField.MONTH_OF_YEAR);
  }
}
```

Converte prima la stringa in minuscolo, perché viene riconosciuto solo **"maggio"**

Il metodo **parse** restituisce un **TemporalAccessor**, da cui occorre estrarre lo specifico campo **ChronoField.MONTH_OF_YEAR**

FORMATTATORI PERSONALIZZATI
Esempio: parsing del nome del mese

- Finalmente, possiamo collaudarlo:

```
System.out.println(
    MyDateUtils.getMonth("Maggio", Locale.ITALY));
```

Java

Risultato: 5

Accettata solo perché il nostro metodo converte prima la stringa in minuscolo

- Volete provare altri paesi e culture?
- OCCHIO, *non sempre i mesi vanno scritti minuscoli!*
  - forse, dopo tutto, incapsulare dentro al metodo una conversione *valida solo per l'italiano* non è stata una grande idea! ☹
  - meglio toglierla e lasciare al chiamante tale onore (onere..)

```
java.time.temporal.TemporalAccessor t =
        f.parse(s.toLowerCase());
```

Co

FORMATTATORI PERSONALIZZATI
Esempio: parsing del nome del mese

- Refactoring (senza conversioni della stringa)

Java

```
public class MyDateUtils {
  public static int getMonth(String s, Locale l){
    DateTimeFormatter f = DateTimeFormatter.ofPattern("MMMM", l);
     java.time.temporal.TemporalAccessor t = f.parse(s);
     return t.get(java.time.temporal.ChronoField.MONTH_OF_YEAR);
  }
}
```

- Nuovi test:

```
System.out.println(
     MyDateUtils.getMonth("mei", Locale.of("nl","NL")));
System.out.println(
     MyDateUtils.getMonth("mai", Locale.FRENCH));
System.out.println(
     MyDateUtils.getMonth("May", Locale.UK));
```

Non esiste una costante predefinita per l'Olanda, occorre costruirla

Costruttore deprecato in Java 19 → usare `of` o `forLanguageTag`

Formattazione di date e orari in C#

FORMATTAZIONE DI DATE E ORARI IN C#

- I principi sono analoghi
- Cultura locale in C# : `System.Globalization.CultureInfo`
- La formattazione di date e orari avviene tramite appositi metodi `ToString` a più argomenti, il primo dei quali è la *stringa di formato*
  - "`d`" data in formato short
  - "`f`" data in formato full (time short)
  - "`g`" data in form. generale (time short)
  - "`t`" orario in formato short
  - … e altre
  - "`D`" data in formato long
  - "`F`" data in formato full (time long)
  - "`G`" data in form. generale (time long)
  - "`T`" orario in formato long

L'eventuale secondo argomento è la *cultura locale.*

FORMATTAZIONE DI DATE E ORARI IN C#

- ESEMPI
  - Formattazione in formato FULL con cultura locale di default

```
Console.WriteLine(today.ToString("F"));
```

  - Formattazione in formato FULL con cultura locale specifica

```
Console.WriteLine(today.ToString("F",itLocale));
```

FORMATTAZIONE DI DATE IN C#: DETTAGLI

```
Console.WriteLine("First day of the week in {0}: {1}",
        CultureInfo.CurrentCulture.DisplayName,
        CultureInfo.CurrentCulture.DateTimeFormat.FirstDayOfWeek.ToString());

Console.WriteLine("{0}",
        String.Join(",", CultureInfo.CurrentCulture.DateTimeFormat.DayNames));

Console.WriteLine("{0}", String.Join(",",
        CultureInfo.CurrentCulture.DateTimeFormat.MonthNames));
```

Primo giorno della settimana

Nomi dei giorni della settimana

Nomi dei mesi

C#

```
Current culture: en-GB
First day of the week in en (GB): Monday
Sunday,Monday,Tuesday,Wednesday,Thursday,Friday,Saturday
January,February,March,April,May,June,July,August,September,October,November,December,
Current culture: fr-CA
First day of the week in fr (CA): Sunday
dimanche,lundi,mardi,mercredi,jeudi,vendredi,samedi
janvier,février,mars,avril,mai,juin,juillet,août,septembre,octobre,novembre,décembre,
Current culture: it-IT
First day of the week in it (IT): Monday
domenica,lunedì,martedì,mercoledì,giovedì,venerdì,sabato
gennaio,febbraio,marzo,aprile,maggio,giugno,luglio,agosto,settembre,ottobre,novembre,dicembre,
Current culture: de-DE
First day of the week in de (DE): Monday
Sonntag,Montag,Dienstag,Mittwoch,Donnerstag,Freitag,Samstag
Januar,Februar,März,April,Mai,Juni,Juli,August,September,Oktober,November,Dezember,
```

C#

FORMATTAZIONE DI DATE E ORARI IN C#: ESEMPI

C#

```
System.Globalization.CultureInfo itLocale =
   new System.Globalization.CultureInfo("it-IT");
DateTime today = DateTime.Today; // date only (time set to midnight)
Console.WriteLine(today.ToString("F"));
Console.WriteLine(today.ToString("F", itLocale));

DateTime now = DateTime.Now;      // date and time
Console.WriteLine(now.ToString("F"));
Console.WriteLine(now.ToString("F", itLocale));
```

F o f

```
Saturday, 20 February 2021 00:00
sabato 20 febbraio 2021 00:00
Saturday, 20 February 2021 00:00:00
sabato 20 febbraio 2021 00:00:00
```

f

Today

```
Saturday, 20 February 2021 18:30
sabato 20 febbraio 2021 18:30
Saturday, 20 February 2021 18:30:15
sabato 20 febbraio 2021 18:30:15
```

Now

F

d = data breve	D = data estesa
f = data completa, ora breve	F = data completa, ora estesa
g = data generale, ora breve	G = data generale, ora estesa
t = ora breve	T = ora estesa
m = mese e giorno	M = mese e giorno
y = mese e anno	Y = mese e anno

FORMATTAZIONE DI DATE E ORARI IN C#: ESEMPI

```
DateTime now = DateTime.Now;        // date and time
Console.WriteLine(now.ToString("d", enGB));
Console.WriteLine(now.ToString("d", itIT));
Console.WriteLine(now.ToString("D", enGB));
Console.WriteLine(now.ToString("D", itIT));
Console.WriteLine(now.ToString("t", enGB));
Console.WriteLine(now.ToString("t", itIT));
Console.WriteLine(now.ToString("T", enGB));
Console.WriteLine(now.ToString("T", itIT));
Console.WriteLine(now.ToString("m", enGB));
Console.WriteLine(now.ToString("m", itIT));
Console.WriteLine(now.ToString("Y", enGB));
Console.WriteLine(now.ToString("Y", itIT));
```

C#

```
20/02/2021
20/02/2021
Saturday, 20 February 2021
sabato 20 febbraio 2021
18:24
18:24
18:24:17
18:24:17
20 February
20 febbraio
February 2021
febbraio 2021
```

- **d** = data breve
- **f** = data completa, ora breve
- **g** = data generale, ora breve
- **t** = ora breve
- **m** o **M** = mese e giorno
- **y** o **Y** = mese e anno
- **D** = data estesa
- **F** = data completa, ora estesa
- **G** = data generale, ora estesa
- **T** = ora estesa

FORMATTAZIONE DI DATE E ORARI IN C#: ESEMPI

```
DateTime xmas2020 = new DateTime(2020, 12, 25);
DateTime xmas2020noon = new DateTime(2020, 12, 25, 12, 0, 0);
Console.WriteLine(xmas2020.ToString("F", itLocale));
Console.WriteLine(xmas2020noon.ToString("F", itLocale));

DateTime xmas2021 = xmas2020.AddYears(1);
DateTime xmas2021noon = xmas2021.AddHours(12);
Console.WriteLine(xmas2021.ToString("F", itLocale));
Console.WriteLine(xmas2021noon.ToString("F", itLocale));
```

Solo giorno

Giorno e ora

C#

```
venerdì 25 dicembre 2020 00:00:00
venerdì 25 dicembre 2020 12:00:00

sabato 25 dicembre 2021 00:00:00
sabato 25 dicembre 2021 12:00:00
```

C#

FORMATTAZIONE DI DATE E ORARI IN C#: ESEMPI

C#

```
TimeZoneInfo localZone = TimeZoneInfo.Local;
TimeZoneInfo utcZone   = TimeZoneInfo.Utc;

Console.WriteLine("Current timezone: {0}", localZone.StandardName);
Console.WriteLine("Daylight name: {0}", localZone.DaylightName);
Console.WriteLine("Bias: {0}", localZone.BaseUtcOffset);
Console.WriteLine("UTC timezone: {0}", utcZone.StandardName);
Console.WriteLine("UTC Daylight name: {0}", utcZone.DaylightName);
Console.WriteLine("UTC Bias: {0}", utcZone.BaseUtcOffset);
```

```
Current timezone: Coordinated Universal Time
Daylight name: Coordinated Universal Time
Bias: 00:00:00
UTC timezone: UTC
UTC Daylight name: UTC
UTC Bias: 00:00:00
```

C#

FORMATTAZIONE DI DATE E ORARI IN C#: ESEMPI

```
DateTime whatTime = today                    // 19 febbraio 2021 00:00:00
                        .AddHours(12)        // 19 febbraio 2021 12:00:00
                        .AddMinutes(10)      // 19 febbraio 2021 12:10:00
                        .AddHours(-13)       // 18 febbraio 2021 23:10:00
                        .AddSeconds(10);     // 18 febbraio 2021 23:10:10
Console.WriteLine(whatTime.ToString("F", itLocale));

DateTimeOffset whatTimeAbsolute =
  new DateTimeOffset(2021, 2, 19, 21, 4, 0, new TimeSpan(1, 0, 0));

Console.WriteLine("whatTimeAbsolute = {0}",
       whatTimeAbsolute.ToString("F", itLocale));
Console.WriteLine("whatTimeAbsolute.UTC = {0}",
       whatTimeAbsolute.UtcDateTime.ToString("F", itLocale));
```

Ore 21:04 in UTC+1

Proprietà UTC di un orario assoluto

C#

```
giovedì 18 febbraio 2021 23:10:10

whatTimeAbsolute = venerdì 19 febbraio 2021 21:04:00
whatTimeAbsolute.UTC = venerdì 19 febbraio 2021 20:04:00
```

Le 21:04 in UTC+1…

…sono le 20:04 UTC

FORMATTAZIONE DI DATE E ORARI IN C#: ESEMPI

```
TimeSpan p1 = new TimeSpan(63,0,0,0);
Console.WriteLine(p1.ToString());

TimeSpan p2 = xmas2021 - xmas2020;
Console.WriteLine(p2.ToString());

DateTime primoAprile = xmas2020.Add(new TimeSpan(97,0,0,0));
Console.WriteLine(primoAprile.ToString("F", itLocale));

DayOfWeek d = (DayOfWeek)1;
Console.WriteLine(d.ToString("F"));    // Monday

Console.WriteLine(DayOfWeek.Tuesday); // Tuesday
```

C#

63 giorni, 0 ore e minuti

1 anno di differenza (365 giorni)

DateTime supporta operatori aritmetici!

97 giorni

1-esimo (2°) giorno della settimana

Costante enumerativa

```
63.00:00:00
365.00:00:00
giovedì 1 aprile 2021 00:00:00
Monday
Tuesday
```

C#

97 giorni dopo Natale

FORMATTAZIONE DI DATE E ORARI IN C#: ESEMPI

C#

```
DateTime startSemester = new DateTime(2021, 2, 17);

Console.WriteLine(startSemester.ToString("ddd"));
Console.WriteLine(startSemester.ToString("ddd", itIT));
Console.WriteLine(startSemester.ToString("dddd"));
Console.WriteLine(startSemester.ToString("dddd", itIT));

Console.WriteLine("xmas2020 = {0}, xmas2021 = {1}, isBefore = {2}",
                  xmas2020.ToString("F", itLocale),
                  xmas2021.ToString("F", itLocale),
                  xmas2020 < xmas2021);
```

Formati di ToString: giorno breve / esteso

```
Wed
mer
Wednesday
mercoledì
```

Su DateTime sono possibili anche operatori relazionali

```
xmas2020 = venerdì 25 dicembre 2020 00:00:00,
xmas2021 = sabato 25 dicembre 2021 00:00:00,
isBefore = True
```

C#

PARSING DI DATE E ORARI IN C#

- In C# , il parsing viene gestito dai metodi statici **`TryParse`**
  - analizzano la stringa e ne determinano *in modo smart* il formato
  - l'output è un argomento *passato per indirizzo (keyword* **`out`***):* il risultato restituito è un **`Bool`** che riporta successo/fallimento
  - ci sono anche i metodi (non statici) **`Parse`** e **`ParseExact`** che però lanciano errore se la stringa non rispetta il formato previsto
- Ci sono fondamentalmente due versioni di **`TryParse`**
  - quella a due argomenti, che usa la cultura predefinita
  - quella a quattro argomenti, che consente di specificare la cultura
- Ci sono metodi **`TryParse`**
  - nei tipi numerici (**`double`**, **`decimal`**, **`int`**…) per parsing di numeri
  - in **`DateTime`**, per date e orari

PARSING DI DATE E ORARI IN C#

Schema base per parsing di date

```
DateTime dateValue;
if (DateTime.TryParse(dateString, out dateValue)) …
if (DateTime.TryParse(dateString, culture, style, out dateValue)) …
```

C#

Passaggio per indirizzo

PARSING DI DATE E ORARI: ESEMPI

C#

```
public class Program
{
    public static void Main() {
      string[] dateStrings = {  "05/01/2009 14:57:32.8", "2009-05-01 14:57:32.8",
                                "2009-05-01T14:57:32.8375298-04:00", "5/01/2008",
                                "5/01/2008 14:57:32.80 -07:00",
                                "1 May 2008 2:57:32.8 PM", "16-05-2009 1:00:32 PM",
                                "Fri, 15 May 2009 20:10:57 GMT" };
      DateTime dateValue;

      Console.WriteLine("Attempting to parse strings using {0} culture:", CultureInfo.CurrentCulture.Name);
      foreach (string dateString in dateStrings) {
         if (DateTime.TryParse(dateString, out dateValue))
            Console.WriteLine("  Converted '{0}' to {1} ({2}).", dateString, dateValue.ToString("F"), dateValue.Kind);
         else
            Console.WriteLine("  Unable to parse '{0}'.", dateString);
      }

      CultureInfo itIT = new CultureInfo("it-IT");
      Console.WriteLine("Tentativo di parsing delle stesse stringhe usando la cultura {0}:", itIT);
      foreach (string dateString in dateStrings) {
         DateTimeStyles style =  DateTimeStyles.None; // OR: AssumeLocal AdjustToUniversal
         if (DateTime.TryParse(dateString, itIT, style, out dateValue))
            Console.WriteLine("  Converted '{0}' to {1} ({2}).", dateString, dateValue.ToString("F", itIT), dateValue.Kind);
         else
            Console.WriteLine("  Unable to parse '{0}'.", dateString);
      }
    }
}
```

PARSING DI DATE E ORARI: ESEMPI

C#

```
string[] dateStrings = {  "05/01/2009 14:57:32.8", "2009-05-01 14:57:32.8",
                          "2009-05-01T14:57:32.8375298-04:00", "5/01/2008",
                          "5/01/2008 14:57:32.80 -07:00",
                          "1 May 2008 2:57:32.8 PM", "16-05-2009 1:00:32 PM",
                          "Fri, 15 May 2009 20:10:57 GMT" };
```

Cultura USA:
16-05-2009 non è interpretabile

```
Attem[illegible] parse strings using en-US culture:
  Co[illegible] '05/01/2009 14:57:32.8' to Friday, May 1, 2009 2:57:32 PM (Unspecified).
  Co[illegible]ed '2009-05-01 14:57:32.8' to Friday, May 1, 2009 2:57:32 PM (Unspecified).
  Co[illegible]ted '2009-05-01T14:57:32.8375298-04:00' to Friday, May 1, 2009 6:57:32 PM (Local).
  Co[illegible]rted '5/01/2008' to Thursday, May 1, 2008 12:00:00 AM (Unspecified).
  Converted '5/01/2008 14:57:32.80 -07:00' to Thursday, May 1, 2008 9:57:32 PM (Local).
  Converted '1 May 2008 2:57:32.8 PM' to Thursday, May 1, 2008 2:57:32 PM (Unspecified).
  Unable to parse '16-05-2009 1:00:32 PM'.
  Converted 'Fri, 15 May 2009 20:10:57 GMT' to Friday, May 15, 2009 8:10:57 PM (Local).

Tentativo di parsing delle stesse stringhe usando la cultura it-IT:
  Converted '05/01/2009 14:57:32.8' to lunedì 5 gennaio 2009 14:57:32 (Unspecified).
  Converted '2009-05-01 14:57:32.8' to venerdì 1 maggio 2009 14:57:32 (Unspecified).
  Converted '2009-05-01T14:57:32.8375298-04:00' to venerdì 1 maggio 2009 18:57:32 (Local).
  Converted '5/01/2008' to sabato 5 gennaio 2008 00:00:00 (Unspecified).
  Converted '5/01/2008 14:57:32.80 -07:00' to sabato 5 gennaio 2008 21:57:32 (Local).
  Converted '1 May 2008 2:57:32.8 PM' to giovedì 1 maggio 2008 14:57:32 (Unspecified).
  Converted '16-05-2009 1:00:32 PM' to sabato 16 maggio 2009 13:00:32 (Unspecified).
  Converted 'Fri, 15 May 2009 20:10:57 GMT' to ven[illegible] 15 maggio 2009 20:10:57 (Local).
```

Cultura USA:
5/1/2009 è il
1° maggio 2009

Cultura Italia:
5/1/2009 è il
5 gennaio 2009

Cultura Italia:
16-05-2009 è correttamente interpretato come il 16 maggio 2009